

NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY POLITEHNICA
BUCHAREST
Faculty of Electronics, Telecommunications and Information Technology

MES Project

Digital Maze Game

Oncioiu Andrei -Daniel AM

2025-2026

Contents

1.	Project description	2
	Project Overview	2
	Core Functionalities.....	2
2.	 Hardware description	
		3
	• 2.2 Hardware Implementation (Schematic)	3
3.	Software description	4
	3.1. Software Architecture	4
	3.2. Peripheral Configuration and Initialization	4
	3.3. Graphical Rendering Logic (ST7735 Driver).....	5
	3.4. Main Game Loop and Logic.....	5
4.	Problems and solutions	5
	4.1. Text Mirroring and Reversed Word Order	5
	4.2. Mirrored Character Rendering (Individual Letters)	6
	4.3. Mechanical Button Bouncing	6
5.	Conclusions.....	6
6.	Source code	7

1. Project description

The project consists of the design and implementation of an **Embedded Maze Game** developed on the **STM32L476RG** (Nucleo) platform. This application integrates hardware control, serial communication (SPI), and real-time game logic to provide an interactive user experience.

Project Overview

The main objective of the game is for the player to navigate a character through a maze, starting from a fixed point and reaching a target destination. The game utilizes an **ST7735 color LCD** (128x128 resolution) for the graphical interface and four physical push-buttons for directional movement.

Core Functionalities

- **Startup Menu:** Upon power-up, the system displays a "START JOC" screen on a yellow background. The game remains in standby until any button is pressed.
- **Maze Navigation:** The player controls the character's movement (Up, Down, Left, Right) using buttons connected to the GPIOB and GPIOC ports.
- **Heads-Up Display (HUD):** A dedicated status bar at the top of the screen tracks and displays the elapsed time in seconds and the total number of steps taken by the player.
- **Audio Feedback:** A piezo buzzer, controlled via bit-banging on the PA10 pin, provides auditory cues for the start of the game and a celebratory melody upon reaching the goal.
- **Victory Condition:** When the player reaches the predefined goal coordinates, the screen turns green and displays "VICTORIE!", and the game enters a finished state.

2. Hardware description

- **2.1 Microcontroller Description: STM32 Nucleo-L476RG**
- The heart of this project is the **STM32L476RG** development board. This microcontroller is part of the ultra-low-power series from STMicroelectronics, based on the high-performance **ARM® Cortex®-M4 32-bit RISC core**.
- **Core Performance:** Operates at a frequency of up to **80 MHz**, providing ample processing power for real-time graphical updates and input monitoring.

- **Memory:** It features **1 MB of Flash memory** (used for code and font storage) and **128 KB of SRAM** (used for runtime variables and stack).
- **Peripherals:**
 - **SPI1:** Utilized for high-speed serial communication with the LCD.
 - **GPIOs:** Configured for digital input (buttons with internal pull-ups) and digital output (LCD control and Buzzer).
 - **Internal RC Oscillator (HSI):** Provides a stable 16 MHz clock source, boosted by the PLL to reach the target operating frequency.

• 2.2 Hardware Implementation (Schematic)

- The hardware setup consists of the Nucleo board, an ST7735 TFT display, four push-buttons, and a piezo buzzer. All components are interfaced using standard jumper wires and a breadboard.

• A. Display Interface (SPI)

- The **ST7735 LCD** uses the **SPI (Serial Peripheral Interface)** protocol. Unlike standard I2C, SPI allows for much higher data transfer rates, which is essential for smooth game animations.

LCD Pin	STM32 Pin	Function	Description
VCC	3.3V	Power	Logic supply voltage.
GND	GND	Ground	Reference voltage.
CS	PA4	Chip Select	Activates the SPI communication for the LCD.
SCK	PA5	Serial Clock	Provides the timing signal for data bits.
SDA (MOSI)	PA7	Master Out	Transmits pixel and command data to the screen.
DC/RS	PB0	Data/Command	Toggles between sending instructions and image data.
RES	PB1	Reset	Hardware reset for the display controller.

• B. User Input (Buttons)

- The control system uses four buttons configured in **Input Pull-Up** mode. This means the pins are internally pulled to 3.3V (Logic 1). When a button is pressed, it connects the pin to Ground (Logic 0), triggering a movement event in the code.
- **Directional Controls:**
 - **PB3 / PB4:** Horizontal movement (Left/Right).
 - **PC8 / PC9:** Vertical movement (Up/Down).

• C. Audio Output (Buzzer)

- The buzzer is connected to pin **PA10**. It operates without a dedicated hardware PWM timer; instead, the software manually toggles the pin high and low (bit-banging) to create the sound frequencies required for the game start and victory melodies.

3. Software description

3.1. Software Architecture

The application is structured into modular layers to separate hardware abstraction from high-level game logic:

- **Hardware Abstraction Layer (HAL):** Manages the configuration of registers for SPI communication and GPIO pins.
- **Display Driver (ST7735):** A custom-built driver that translates high-level drawing commands (like strings or rectangles) into SPI data packets.
- **Game Logic:** Handles the character movement, collision detection, and game state transitions.

3.2. Peripheral Configuration and Initialization

At startup, the system executes several initialization routines:

- **Clock Configuration:** The `SystemClock_Config()` function sets the processor speed to ensure stable SPI communication and timing.
- **SPI Initialization:** Configures the SPI1 peripheral in Master mode with an 8-bit data size and MSB-first bit order, matching the ST7735 controller requirements.
- **GPIO Initialization:** Defines the input/output roles for each pin. Buttons are initialized with internal **pull-up resistors** to ensure a stable high state when not pressed.

3.3. Graphical Rendering Logic (ST7735 Driver)

Since the LCD does not have built-in font support, the software renders text and graphics manually:

- **Window Addressing:** The `ST7735_SetAddressWindow()` function defines the specific coordinates on the screen where pixels will be drawn, optimizing data transfer.
- **Font Rendering:** Characters are stored as bitmaps in a font file. The `ST7735_DrawChar()` function reads these bitmaps bit by bit. To ensure characters appear correctly (not mirrored), the logic scans bits from the most significant position to the least significant position.
- **Color Format:** The display operates in **RGB565** format, where each pixel color is represented by a 16-bit value (5 bits for Red, 6 for Green, 5 for Blue).

3.4. Main Game Loop and Logic

The core of the application resides in an infinite `while(1)` loop in `main.c`, which manages three main tasks:

A. Non-blocking HUD Updates

Instead of using `HAL_Delay()` (which would freeze the game), the code uses `HAL_GetTick()`. Every 1000 milliseconds, the `Update_HUD()` function is called to refresh the timer on the screen and update the step counter.

B. Input Handling and Debouncing

The software constantly monitors the GPIO pins for the buttons. To prevent "bouncing" (a single press being registered as multiple moves), a temporal window of 120ms (DEBOUNCE_MS) is implemented. A movement is only processed if enough time has passed since the last valid input.

C. Game State and Victory Condition

The game tracks the player's coordinates (player_x, player_y). After every move, Maze_CheckWin() compares the player's position with the goal's location. If they match, the game enters the "Victory" state, displays the success message, and plays a musical sequence via the buzzer.

4. Problems and solutions

During the integration of the ST7735 LCD driver and the user input logic, three primary issues were identified and corrected.

4.1. Text Mirroring and Reversed Word Order

- **The Problem:** Upon the initial implementation of the ST7735_WriteString function, the intended text "START JOC" appeared on the screen as "coj trats". The words were being written from right to left instead of left to right.
- **The Cause:** The issue was traced to the MADCTL (Memory Data Access Control) register of the ST7735 controller. The bit responsible for horizontal mirroring (MX) was inadvertently set to 1 in the rotation configuration.
- **The Solution:** The ST7735_SetRotation function was modified. By clearing the MADCTL_MX bit, the internal memory pointer of the display was instructed to increment from left to right, restoring the natural reading order.

4.2. Mirrored Character Rendering (Individual Letters)

- **The Problem:** After fixing the word order, a secondary issue persisted where the characters themselves were mirrored (e.g., the letter 'S' appeared as 'Ɔ'). While the cursor moved in the correct direction, the pixel data for each letter was rendered backwards.
- **The Cause:** This was a logical error in the ST7735_DrawChar function. The font bitmaps were stored such that the first pixel of a row corresponded to the most significant bit (MSB), but the code was reading the least significant bit (LSB) first.
- **The Solution:** The nested loop responsible for drawing the character's width was updated. Instead of using a simple shift ($b \gg j$), the logic was changed to ($b \gg (font.width - 1 - j)$). This ensured that the bits were read from MSB to LSB, aligning the font data correctly with the screen's X-axis.

4.3. Mechanical Button Bouncing

- **The Problem:** The player character often moved two or three tiles with a single, quick button press. This made the game nearly unplayable as precise navigation through narrow corridors was impossible.
- **The Cause:** Mechanical buttons do not provide a clean electrical signal; when pressed, the internal metal leaves "bounce," creating multiple rapid pulses that the high-speed STM32 processor interprets as multiple distinct presses.
- **The Solution:** A **software debounce** mechanism was implemented. A global constant `DEBOUNCE_MS` was set to 120ms. The code now captures the current time using `HAL_GetTick()` and only allows a new movement if the difference between the current time and the `lastMoveTime` exceeds this threshold.

5. Conclusions

- The development of the **STM32 Maze Game** successfully integrated hardware peripherals with high-level software logic to create a functional and engaging embedded application.
- The project highlights several key takeaways in embedded systems engineering:
- **Protocol Mastery:** The successful implementation of the **SPI protocol** ensured high-speed data transmission, which is critical for maintaining a responsive graphical user interface on the ST7735 LCD.
- **Low-Level Debugging:** Resolving the "coj trats" mirroring issue provided a deep understanding of how display controllers interpret memory data and the importance of precise register configuration (specifically the **MADCTL** register).
- **Resource Management:** By using non-blocking timing methods (`HAL_GetTick()`) instead of processor-halting delays, the system remained efficient, allowing for simultaneous input monitoring, HUD updates, and audio output.
- **Hardware-Software Synergy:** The project demonstrates how software can compensate for hardware limitations, such as using **software debouncing** to clean mechanical switch signals and using **bit-banging** to generate audio frequencies without dedicated PWM hardware.
- Overall, the project achieved all initial objectives, resulting in a stable and playable game that serves as a robust foundation for more complex embedded graphical applications.

6. Source code

MAIN.C

```
#include "st7735.h"
```

```

#include "maze.h"
#include "player.h"
#include "stm32l4xx_hal.h"
#include <stdlib.h>
#include <stdio.h>

extern FontDef Font_7x10;

/* DEFINIȚII CULORI */
#define YELLOW 0x07FF
#define GREEN 0x07E0
#define BLACK 0x0000
#define WHITE 0xFFFF
#define BLUE 0x001F
#define RED 0xF800

SPI_HandleTypeDef hspi1;
extern uint8_t player_x, player_y;

// --- Variabila pentru nivel ---
uint8_t currentLevel = 0;
#define MAX_LEVELS 3

void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_SPI1_Init(void);

// Functie simplă de delay
void Delay_Soft(uint32_t count) {
while(count--) { __NOP(); __NOP(); __NOP(); __NOP(); }
}

// Functie pentru sunet
void Play_Tone(uint32_t pitch, uint32_t duration_ms) {
volatile uint32_t * const gpioa_bsrr = (volatile uint32_t *)0x48000018;
uint32_t cycles = (duration_ms * 1000) / (pitch * 2);

for (uint32_t i = 0; i < cycles; i++) {
*gpioa_bsrr = 0x400;
Delay_Soft(pitch);
*gpioa_bsrr = 0x4000000;
Delay_Soft(pitch);
*gpioa_bsrr = 0x400;
}
}

// Functie afișare scor/timp
void Update_HUD(uint32_t time_sec, int steps, int level) {
char buffer[20];
// Curățăm zona de sus

```



```

ST7735_FillRect(0, 0, 160, 12, BLACK);

// Afisăm Timpul
sprintf(buffer, "T:%lu", time_sec);
ST7735_WriteString(2, 1, buffer, Font_7x10, WHITE, BLACK);

// Afisăm Pasii
sprintf(buffer, "S:%d", steps);
ST7735_WriteString(50, 1, buffer, Font_7x10, YELLOW, BLACK);

// Afisăm Nivelul (L:1, L:2 etc)
sprintf(buffer, "L:%d", level + 1);
ST7735_WriteString(95, 1, buffer, Font_7x10, GREEN, BLACK);
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_SPI1_Init();

    ST7735_Init();

    // Ecran de Start
    ST7735_FillScreen(YELLOW);
    ST7735_WriteString(10, 40, "START JOC", Font_7x10, BLACK, YELLOW);

    Play_Tone(1000, 100);

    // Așteptăm apăsarea unui buton
    while (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_3) == GPIO_PIN_SET &&
        HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_4) == GPIO_PIN_SET &&
        HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_8) == GPIO_PIN_SET &&
        HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_9) == GPIO_PIN_SET) {}

    srand(HAL_GetTick());
    ST7735_FillScreen(BLACK);

    // --- INITIALIZARE PRIMUL NIVEL ---
    currentLevel = 0;
    Maze_LoadLevel(currentLevel); // Încarcă harta 0 și generează finish
    Maze_Draw();
    Player_Init(1, 1);

    uint32_t startTime = HAL_GetTick();
    uint32_t lastHudUpdate = 0;
    int steps = 0;
    const uint32_t DEBOUNCE_MS = 120; // Viteza de mișcare
    uint32_t lastMoveTime = 0;

```

```

while(1)
{
    uint32_t now = HAL_GetTick();

    // Update HUD la fiecare secundă
    if (now - lastHudUpdate >= 1000) {
        Update_HUD((now - startTime) / 1000, steps, currentLevel);
        lastHudUpdate = now;
    }

    // Citire Butoane cu Debounce
    if(now - lastMoveTime > DEBOUNCE_MS)
    {
        int moved = 0;

        if(HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_3) == GPIO_PIN_RESET) { Player_Move('d');
        moved = 1; }
        if(HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_4) == GPIO_PIN_RESET) { Player_Move('s');
        moved = 1; }
        if(HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_8) == GPIO_PIN_RESET) { Player_Move('w');
        moved = 1; }
        if(HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_9) == GPIO_PIN_RESET) { Player_Move('a');
        moved = 1; }

        if(moved)
        {
            lastMoveTime = now;
            steps++;
            Update_HUD((now - startTime)/1000, steps, currentLevel);

            // --- VERIFICARE CÂȘTIG ---
            if (Maze_CheckWin(player_x, player_y))
            {
                // Sunet victorie nivel
                Play_Tone(600, 100);
                Play_Tone(800, 100);

                // Treceam la nivelul următor
                currentLevel++;

                if (currentLevel < MAX_LEVELS)
                {
                    // --- CAZ: MAI SUNT NIVELE ---

                    // Mesaj scurt
                    ST7735_FillScreen(BLACK);
                    ST7735_WriteString(10, 60, "NIVEL COMPLET!", Font_7x10, GREEN, BLACK);
                    HAL_Delay(1500);
                }
            }
        }
    }
}

```

```

// Resetare pentru noul nivel
ST7735_FillScreen(BLACK);
Maze_LoadLevel(currentLevel); // Încarcă harta nouă
Maze_Draw(); // Desenează peretii
Player_Init(1, 1); // Pune jucătorul la start

// NOTĂ: Nu resetăm timpul sau pașii (lăsăm să se acumuleze)
// Dacă vrei să resetezi pașii, pune: steps = 0; aici.
}
else
{
// --- CAZ: JOC TERMINAT (FINAL) ---
ST7735_FillScreen(GREEN);
ST7735_WriteString(20, 40, "FELICITARI!", Font_7x10, BLACK, GREEN);
ST7735_WriteString(20, 60, "AI TERMINAT", Font_7x10, BLACK, GREEN);
ST7735_WriteString(20, 80, "TOATE NIVELELE", Font_7x10, BLACK, GREEN);

// Melodie finală
Play_Tone(500, 150);
Play_Tone(600, 150);
Play_Tone(700, 150);
Play_Tone(800, 400);

while(1); // Blochează jocul aici
}
}
}
}
}
}

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
RCC_OscInitTypeDef RCC_OscInitStruct = {0};
RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

/** Configure the main internal regulator output voltage
 */
if (HAL_PWREx_ControlVoltageScaling(PWR_REGULATOR_VOLTAGE_SCALE1) != HAL_OK)
{
Error_Handler();
}

/** Initializes the RCC Oscillators according to the specified parameters
 * in the RCC_OscInitTypeDef structure.
 */
RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;

```

```

RCC_OscInitStruct.HSISate = RCC_HSI_ON;
RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
RCC_OscInitStruct.PLL.PLLM = 1;
RCC_OscInitStruct.PLL.PLLN = 10;
RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV7;
RCC_OscInitStruct.PLL.PLLQ = RCC_PLLQ_DIV2;
RCC_OscInitStruct.PLL.PLLR = RCC_PLLR_DIV2;
if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
{
    Error_Handler();
}

/** Initializes the CPU, AHB and APB buses clocks
 */
RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYCLK
|RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_4) != HAL_OK)
{
    Error_Handler();
}
}

/**
 * @brief SPI1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_SPI1_Init(void)
{

    /* USER CODE BEGIN SPI1_Init 0 */

    /* USER CODE END SPI1_Init 0 */

    /* USER CODE BEGIN SPI1_Init 1 */

    /* USER CODE END SPI1_Init 1 */
    /* SPI1 parameter configuration*/
    hspi1.Instance = SPI1;
    hspi1.Init.Mode = SPI_MODE_MASTER;
    hspi1.Init.Direction = SPI_DIRECTION_2LINES;
    hspi1.Init.DataSize = SPI_DATASIZE_8BIT;
    hspi1.Init.CLKPolarity = SPI_POLARITY_LOW;
    hspi1.Init.CLKPhase = SPI_PHASE_1EDGE;

```

```

hspi1.Init.NSS = SPI_NSS_SOFT;
hspi1.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_16;
hspi1.Init.FirstBit = SPI_FIRSTBIT_MSB;
hspi1.Init.TIMode = SPI_TIMODE_DISABLE;
hspi1.Init.CRCCalculation = SPI_CRCCALCULATION_DISABLE;
hspi1.Init.CRCPolynomial = 7;
hspi1.Init.CRCLength = SPI_CRC_LENGTH_DATASIZE;
hspi1.Init.NSSPMode = SPI_NSS_PULSE_ENABLE;
if (HAL_SPI_Init(&hspi1) != HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN SPI1_Init 2 */

/* USER CODE END SPI1_Init 2 */

}

/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    /* USER CODE BEGIN MX_GPIO_Init_1 */
    /* USER CODE END MX_GPIO_Init_1 */

    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOC_CLK_ENABLE();
    __HAL_RCC_GPIOH_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_3, GPIO_PIN_RESET);

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_SET);

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0|GPIO_PIN_1, GPIO_PIN_RESET);

    /*Configure GPIO pin : B1_Pin */
    GPIO_InitStruct.Pin = GPIO_PIN_1;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;

```

```

HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

/*Configure GPIO pin : PC3 */
GPIO_InitStruct.Pin = GPIO_PIN_3;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

/*Configure GPIO pin : PA4 */
GPIO_InitStruct.Pin = GPIO_PIN_4|GPIO_PIN_10;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

/*Configure GPIO pins : PB0 PB1 */
GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

/*Configure GPIO pins : PC8 PC9 */
GPIO_InitStruct.Pin = GPIO_PIN_8|GPIO_PIN_9;
GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
GPIO_InitStruct.Pull = GPIO_PULLUP;
HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

/*Configure GPIO pins : PB3 PB4 */
GPIO_InitStruct.Pin = GPIO_PIN_3|GPIO_PIN_4;
GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
GPIO_InitStruct.Pull = GPIO_PULLUP;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);


/* USER CODE BEGIN MX_GPIO_Init_2 */
/* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */

/* USER CODE END 4 */

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */

```

```

void Error_Handler(void)
{
/* USER CODE BEGIN Error_Handler_Debug */
/* User can add his own implementation to report the HAL error return state */
__disable_irq();
while (1)
{
}
/* USER CODE END Error_Handler_Debug */
}

#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number
 * where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
/* USER CODE BEGIN 6 */
/* User can add his own implementation to report the file name and line number,
ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
/* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */

```

FONTS.C

```

#include "fonts.h"

/* Font 7x10 (Lățime 7, Înălțime 10)
Datele sunt aranjate pe rânduri. Bitul 0 = Pixel Stânga.
*/
const uint16_t Font7x10_Data [] = {
// SPAȚIU (32)
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
// ! (33)
0x04, 0x04, 0x04, 0x04, 0x04, 0x04, 0x00, 0x04, 0x00, 0x00,
// "
0x14, 0x14, 0x14, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
// #
0x00, 0x14, 0x14, 0x3E, 0x14, 0x3E, 0x14, 0x14, 0x00, 0x00,
// $
0x04, 0x1E, 0x24, 0x14, 0x0E, 0x05, 0x24, 0x3C, 0x04, 0x00,
// %
0x00, 0x23, 0x13, 0x08, 0x04, 0x02, 0x32, 0x31, 0x00, 0x00,
// &
0x00, 0x0C, 0x12, 0x12, 0x0C, 0x2A, 0x12, 0x2D, 0x00, 0x00,
// '

```

```

0x04, 0x04, 0x08, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
// (
0x00, 0x02, 0x04, 0x08, 0x08, 0x08, 0x04, 0x02, 0x00, 0x00,
// )
0x00, 0x10, 0x08, 0x04, 0x04, 0x04, 0x08, 0x10, 0x00, 0x00,
// *
0x00, 0x04, 0x15, 0x0E, 0x15, 0x04, 0x00, 0x00, 0x00, 0x00,
// +
0x00, 0x04, 0x04, 0x1F, 0x04, 0x04, 0x00, 0x00, 0x00, 0x00,
// ,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0C, 0x04, 0x02, 0x00,
// -
0x00, 0x00, 0x00, 0x00, 0x1F, 0x00, 0x00, 0x00, 0x00, 0x00,
// .
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0C, 0x0C, 0x00,
// /
0x00, 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x00, 0x00, 0x00,

// 0-9
0x00, 0x0E, 0x11, 0x13, 0x15, 0x19, 0x11, 0x0E, 0x00, 0x00, // 0
0x00, 0x04, 0x0C, 0x04, 0x04, 0x04, 0x04, 0x0E, 0x00, 0x00, // 1
0x00, 0x0E, 0x11, 0x01, 0x02, 0x04, 0x08, 0x1F, 0x00, 0x00, // 2
0x00, 0x1F, 0x02, 0x04, 0x02, 0x01, 0x11, 0x0E, 0x00, 0x00, // 3
0x00, 0x02, 0x06, 0x0A, 0x12, 0x1F, 0x02, 0x02, 0x00, 0x00, // 4
0x00, 0x1F, 0x10, 0x1E, 0x01, 0x01, 0x11, 0x0E, 0x00, 0x00, // 5
0x00, 0x06, 0x08, 0x10, 0x1E, 0x11, 0x11, 0x0E, 0x00, 0x00, // 6
0x00, 0x1F, 0x01, 0x02, 0x04, 0x08, 0x08, 0x08, 0x00, 0x00, // 7
0x00, 0x0E, 0x11, 0x11, 0x0E, 0x11, 0x11, 0x0E, 0x00, 0x00, // 8
0x00, 0x0E, 0x11, 0x11, 0x0F, 0x01, 0x02, 0x0C, 0x00, 0x00, // 9

// :
0x00, 0x00, 0x04, 0x00, 0x00, 0x04, 0x00, 0x00, 0x00, 0x00,
// ;
0x00, 0x00, 0x04, 0x00, 0x00, 0x04, 0x02, 0x01, 0x00, 0x00,
// <
0x00, 0x02, 0x04, 0x08, 0x10, 0x08, 0x04, 0x02, 0x00, 0x00,
// =
0x00, 0x00, 0x00, 0x1F, 0x00, 0x1F, 0x00, 0x00, 0x00, 0x00,
// >
0x00, 0x10, 0x08, 0x04, 0x02, 0x04, 0x08, 0x10, 0x00, 0x00,
// ?
0x00, 0x0E, 0x11, 0x01, 0x02, 0x04, 0x00, 0x04, 0x00, 0x00,
// @
0x00, 0x0E, 0x11, 0x01, 0x0D, 0x15, 0x15, 0x0E, 0x00, 0x00,

// A-Z (Majuscule)
0x00, 0x04, 0x0A, 0x11, 0x11, 0x1F, 0x11, 0x11, 0x00, 0x00, // A
0x00, 0x1E, 0x11, 0x11, 0x1E, 0x11, 0x11, 0x1E, 0x00, 0x00, // B
0x00, 0x0E, 0x11, 0x10, 0x10, 0x10, 0x11, 0x0E, 0x00, 0x00, // C
0x00, 0x1E, 0x11, 0x11, 0x11, 0x11, 0x11, 0x1E, 0x00, 0x00, // D
0x00, 0x1F, 0x10, 0x10, 0x1C, 0x10, 0x10, 0x1F, 0x00, 0x00, // E

```



```

0x00, 0x1F, 0x10, 0x10, 0x1C, 0x10, 0x10, 0x10, 0x00, 0x00, // F
0x00, 0x0E, 0x11, 0x10, 0x17, 0x11, 0x11, 0x0F, 0x00, 0x00, // G
0x00, 0x11, 0x11, 0x11, 0x1F, 0x11, 0x11, 0x11, 0x00, 0x00, // H
0x00, 0x0E, 0x04, 0x04, 0x04, 0x04, 0x04, 0x0E, 0x00, 0x00, // I
0x00, 0x07, 0x02, 0x02, 0x02, 0x02, 0x12, 0x0C, 0x00, 0x00, // J
0x00, 0x11, 0x12, 0x14, 0x18, 0x14, 0x12, 0x11, 0x00, 0x00, // K
0x00, 0x10, 0x10, 0x10, 0x10, 0x10, 0x10, 0x1F, 0x00, 0x00, // L
0x00, 0x11, 0x1B, 0x15, 0x15, 0x11, 0x11, 0x11, 0x00, 0x00, // M
0x00, 0x11, 0x11, 0x19, 0x15, 0x13, 0x11, 0x11, 0x00, 0x00, // N
0x00, 0x0E, 0x11, 0x11, 0x11, 0x11, 0x11, 0x0E, 0x00, 0x00, // O
0x00, 0x1E, 0x11, 0x11, 0x1E, 0x10, 0x10, 0x10, 0x00, 0x00, // P
0x00, 0x0E, 0x11, 0x11, 0x11, 0x15, 0x12, 0x0D, 0x00, 0x00, // Q
0x00, 0x1E, 0x11, 0x11, 0x1E, 0x14, 0x12, 0x11, 0x00, 0x00, // R
0x00, 0x0E, 0x11, 0x10, 0x0E, 0x01, 0x11, 0x0E, 0x00, 0x00, // S
0x00, 0x1F, 0x04, 0x04, 0x04, 0x04, 0x04, 0x04, 0x00, 0x00, // T
0x00, 0x11, 0x11, 0x11, 0x11, 0x11, 0x11, 0x0E, 0x00, 0x00, // U
0x00, 0x11, 0x11, 0x11, 0x11, 0x11, 0x0A, 0x04, 0x00, 0x00, // V
0x00, 0x11, 0x11, 0x11, 0x15, 0x15, 0x1B, 0x11, 0x00, 0x00, // W
0x00, 0x11, 0x11, 0x0A, 0x04, 0x0A, 0x11, 0x11, 0x00, 0x00, // X
0x00, 0x11, 0x11, 0x0A, 0x04, 0x04, 0x04, 0x04, 0x00, 0x00, // Y
0x00, 0x1F, 0x01, 0x02, 0x04, 0x08, 0x10, 0x1F, 0x00, 0x00, // Z

// a-z și restul (umplutură simplă pentru a nu crăpa la litere mici)
// Mapăm literele mici la fel ca cele mari pentru simplitate
// (sau poți da copy-paste la A-Z de mai sus de încă 2 ori)
};

/* Structura Font */
FontDef Font_7x10 = {7, 10, Font7x10_Data};
FontDef Font_11x18 = {7, 10, Font7x10_Data}; // Folosim acelasi font pt sigurantă

```

MAZE.C

```

#include "maze.h"
#include <stdlib.h>
#include <string.h> // Necesari pentru memcpy

#ifndef BLUE
#define BLUE 0x001F
#endif
#ifndef BLACK
#define BLACK 0x0000
#endif
#ifndef RED
#define RED 0xF800
#endif
#ifndef GREEN
#define GREEN 0x07E0
#endif

```

```

// Definim numărul de nivele disponibile
#define TOTAL_LEVELS 3

// Matricea activă (pe care se joacă efectiv)
uint8_t maze[MAZE_H][MAZE_W];

// Baza de date cu hărțile nivelelor (Constantă - nu se modifică în timpul
jocului)
// Nivelul 0 = Originalul tău
// Nivelul 1 = Design tip Zig-Zag
// Nivelul 2 = Design tip Camere/Insule
const uint8_t maps[TOTAL_LEVELS][MAZE_H][MAZE_W] =
{
// --- NIVELUL 1 ---
{
{1,1,1,1,1,1,1,1,1,1,1,1,1,1,1},
{1,0,0,0,1,0,0,0,0,0,1,0,0,0,1},
{1,0,1,0,1,0,1,1,1,0,1,0,1,1,0,1},
{1,0,1,0,0,0,0,0,1,0,0,0,0,1,0,1},
{1,0,1,1,1,1,1,0,1,1,1,1,0,1,0,1},
{1,0,0,0,0,0,1,0,0,0,0,1,0,0,0,1},
{1,1,1,1,1,0,1,1,1,1,0,1,1,1,0,1},
{1,0,0,0,1,0,0,0,0,1,0,0,0,1,0,1},
{1,0,1,0,1,1,1,1,0,1,1,1,0,1,0,1},
{1,0,1,0,0,0,0,1,0,0,0,1,0,0,0,1},
{1,0,1,1,1,1,0,1,1,1,0,1,1,1,0,1},
{1,0,0,0,0,1,0,0,0,1,0,0,0,1,0,1},
{1,1,1,1,0,1,1,1,0,1,1,1,0,1,0,1},
{1,0,0,1,0,0,0,1,0,0,0,1,0,0,0,1},
{1,0,1,1,1,1,0,1,1,1,0,1,1,1,0,1},
{1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1},
{1,1,1,1,1,1,1,1,1,1,1,1,1,1,1}
},

// --- NIVELUL 2 ---
{
{1,1,1,1,1,1,1,1,1,1,1,1,1,1,1},
{1,0,0,0,0,0,0,0,0,1,0,0,0,0,1},
{1,1,1,1,1,0,1,1,0,1,0,1,1,1,0,1},
{1,0,0,0,1,0,1,0,0,0,0,1,0,0,0,1},
{1,0,1,0,1,0,1,1,1,1,1,1,0,1,1,1},
{1,0,1,0,0,0,0,0,0,0,0,0,0,0,0,1},
{1,0,1,1,1,1,1,0,1,1,1,0,1,1,0,1},
{1,0,0,0,0,0,1,0,1,0,0,0,1,0,0,1},
{1,1,1,1,1,0,1,0,1,0,1,1,1,0,1,1},
{1,0,0,0,0,0,1,0,0,0,0,0,1,0,0,1},
{1,0,1,1,1,1,1,1,1,1,1,0,1,1,0,1},
{1,0,1,0,0,0,0,0,0,0,0,0,0,0,0,1},
{1,0,1,0,1,1,1,1,0,1,1,1,1,1,0,1},
{1,0,0,0,1,0,0,0,0,1,0,0,0,0,0,1},
{1,1,1,0,1,0,1,1,1,1,0,1,1,1,1,1},

```

```

{1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1},
{1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1},
},

// --- NIVELUL 3 ---
{
{1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1},
{1,0,1,0,0,0,0,1,0,0,0,0,0,0,0,1},
{1,0,1,0,1,1,0,1,0,1,1,1,1,1,0,1},
{1,0,0,0,1,0,0,0,0,0,0,0,0,1,0,1},
{1,1,1,0,1,1,1,1,1,1,1,1,0,1,0,1},
{1,0,0,0,0,0,1,0,0,0,0,1,0,0,0,1},
{1,0,1,1,1,0,1,0,1,1,0,1,1,1,1,1},
{1,0,1,0,0,0,0,0,1,0,0,0,0,0,0,1},
{1,0,1,0,1,1,1,0,1,0,1,1,1,0,1,1},
{1,0,0,0,1,0,0,0,1,0,1,0,0,0,0,1},
{1,1,1,1,1,0,1,1,1,0,1,0,1,1,1,1},
{1,0,0,0,0,0,0,0,0,0,1,0,0,0,0,1},
{1,0,1,1,1,1,1,1,1,1,1,1,1,1,0,1},
{1,0,0,0,0,0,1,0,0,0,0,0,0,0,0,1},
{1,1,1,1,1,0,1,1,1,0,1,0,1,1,1,1},
{1,0,0,0,0,0,0,0,0,0,1,0,0,0,0,1},
{1,0,1,1,1,1,1,1,1,1,1,1,1,1,0,1},
{1,0,0,0,0,0,1,0,0,0,0,0,0,0,0,1},
{1,1,1,1,1,0,1,0,1,1,1,1,1,0,1,1},
{1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1},
{1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1}
}
};

void Maze_GenerateGoal(void)
{
uint8_t placed = 0;
while (!placed)
{
// Se alege un punct random
uint8_t rx = rand() % MAZE_W;
uint8_t ry = rand() % MAZE_H;

// Verificăm să fie spațiu liber (0) și să NU fie colțul de start (1,1)
if (maze[ry][rx] == 0 && (rx != 1 || ry != 1))
{
maze[ry][rx] = 2; // Punem 2 (Goal)
placed = 1;
}
}
}

// Functie nouă: Încarcă un nivel din memoria constantă în memoria activă
// Parametrul levelIndex trebuie să fie între 0 și TOTAL_LEVELS - 1
void Maze_LoadLevel(uint8_t levelIndex)
{
if (levelIndex >= TOTAL_LEVELS) return; // Siguranță

// Copiem harta curată din 'maps' în 'maze'

```

```

memcpy(maze, maps[levelIndex], sizeof(maze));

// Generăm un nou punct de finish pentru acest nivel
Maze_GenerateGoal();
}

void Maze_Draw(void)
{
for (uint8_t y = 0; y < MAZE_H; y++)
{
for (uint8_t x = 0; x < MAZE_W; x++)
{
uint16_t color;
if (maze[y][x] == 1) color = BLUE; // Perete
else if (maze[y][x] == 2) color = RED; // Destinație
else color = BLACK; // Culoar

ST7735_FillRect(
MAZE_OFFSET_X + (x * CELL_SIZE),
MAZE_OFFSET_Y + (y * CELL_SIZE),
CELL_SIZE,
CELL_SIZE,
color
);
}
}
}

uint8_t Maze_CheckWin(uint8_t px, uint8_t py)
{
// Verifică limitele hărtii
if (px >= MAZE_W || py >= MAZE_H) return 0;

// Dacă jucătorul este pe celulă cu valoarea 2 (Goal)
if (maze[py][px] == 2)
{
ST7735_FillScreen(GREEN);
return 1;
}
return 0;
}

```

PLAYER.C

```

#include "player.h"
#include "maze.h"
#include "st7735.h"

/* Coordonate globale */
uint8_t player_x = 1;

```

```

uint8_t player_y = 1;

/* Configurare vizuală */
#define PLAYER_COLOR 0xF81F // Magenta

void Player_Init(uint8_t startX, uint8_t startY)
{
    player_x = startX;
    player_y = startY;

    // Desenare inițială cu OFFSET (Important!)
    ST7735_FillRect(
        MAZE_OFFSET_X + (player_x * CELL_SIZE),
        MAZE_OFFSET_Y + (player_y * CELL_SIZE),
        CELL_SIZE,
        CELL_SIZE,
        PLAYER_COLOR
    );
}

void Player_Move(char direction)
{
    uint8_t new_x = player_x;
    uint8_t new_y = player_y;

    // Calculăm noua poziție
    switch(direction)
    {
        case 'w': new_y--; break; // Sus
        case 's': new_y++; break; // Jos
        case 'a': new_x--; break; // Stânga
        case 'd': new_x++; break; // Dreapta
    }

    // Verificăm limitele matricei
    if (new_x >= MAZE_W || new_y >= MAZE_H) return;

    // Verificăm coliziunea: Permite doar dacă NU e perete (1)
    // 0 = Drum, 2 = Final. Ambele sunt accesibile.
    if (maze[new_y][new_x] != 1)
    {
        // 1. Stergem poziția veche (desenăm Negru)
        ST7735_FillRect(
            MAZE_OFFSET_X + (player_x * CELL_SIZE),
            MAZE_OFFSET_Y + (player_y * CELL_SIZE),
            CELL_SIZE,
            CELL_SIZE,
            BLACK
        );
    }
}

```

```
// 2. Actualizăm coordonatele
player_x = new_x;
player_y = new_y;

// 3. Desenăm pe poziția nouă
ST7735_FillRect(
MAZE_OFFSET_X + (player_x * CELL_SIZE),
MAZE_OFFSET_Y + (player_y * CELL_SIZE),
CELL_SIZE,
CELL_SIZE,
PLAYER_COLOR
);
}
}
```

ST7735.C

```
#include "st7735.h"

extern SPI_HandleTypeDef hspi1;

#define CS_PORT GPIOA
#define CS_PIN GPIO_PIN_4
#define DC_PORT GPIOB
#define DC_PIN GPIO_PIN_0
#define RST_PORT GPIOB
#define RST_PIN GPIO_PIN_1

uint8_t colstart = 0;
uint8_t rowstart = 0;
uint8_t _width = 128;
uint8_t _height = 128;

#define CS_LOW() HAL_GPIO_WritePin(CS_PORT, CS_PIN, GPIO_PIN_RESET)
#define CS_HIGH() HAL_GPIO_WritePin(CS_PORT, CS_PIN, GPIO_PIN_SET)
#define DC_CMD() HAL_GPIO_WritePin(DC_PORT, DC_PIN, GPIO_PIN_RESET)
#define DC_DATA() HAL_GPIO_WritePin(DC_PORT, DC_PIN, GPIO_PIN_SET)

#define MADCTL 0x36
#define MADCTL_MY 0x80
#define MADCTL_MX 0x40
#define MADCTL_MV 0x20
#define MADCTL_ML 0x10
#define MADCTL_RGB 0x00
#define MADCTL_BGR 0x08
#define MADCTL_MH 0x04

static void ST7735_WriteCommand(uint8_t cmd) {
```

```

DC_CMD();
CS_LOW();
HAL_SPI_Transmit(&hspi1, &cmd, 1, 100);
CS_HIGH();
}

static void ST7735_WriteData(uint8_t *buff, size_t buff_size) {
DC_DATA();
CS_LOW();
HAL_SPI_Transmit(&hspi1, buff, buff_size, 100);
CS_HIGH();
}

static void ST7735_SetAddressWindow(uint16_t x0, uint16_t y0, uint16_t x1,
uint16_t y1) {

uint16_t x_start = x0 + colstart;
uint16_t x_end = x1 + colstart;
uint16_t y_start = y0 + rowstart;
uint16_t y_end = y1 + rowstart;

uint8_t data[4];
ST7735_WriteCommand(0x2A);
data[0] = x_start >> 8; data[1] = x_start & 0xFF;
data[2] = x_end >> 8; data[3] = x_end & 0xFF;
ST7735_WriteData(data, 4);

ST7735_WriteCommand(0x2B); // RASET
data[0] = y_start >> 8; data[1] = y_start & 0xFF;
data[2] = y_end >> 8; data[3] = y_end & 0xFF;
ST7735_WriteData(data, 4);

ST7735_WriteCommand(0x2C); // RAMWR
}

void ST7735_Init(void) {
HAL_GPIO_WritePin(RST_PORT, RST_PIN, GPIO_PIN_RESET);
HAL_Delay(50);
HAL_GPIO_WritePin(RST_PORT, RST_PIN, GPIO_PIN_SET);
HAL_Delay(150);

ST7735_WriteCommand(0x01); // SWRESET
HAL_Delay(150);
ST7735_WriteCommand(0x11); // SLPOUT
HAL_Delay(200);

uint8_t color_mode = 0x05; // RGB565
ST7735_WriteCommand(0x3A);
ST7735_WriteData(&color_mode, 1);

```

```

ST7735_WriteCommand(0x29); // DISPON
HAL_Delay(10);
}

void ST7735_SetRotation(uint8_t m) {
ST7735_WriteCommand(MADCTL);
uint8_t rotation;

switch (m) {
case 2:
/* AM ADĂUGAT 'MADCTL_MX' PENTRU A REPARA OGINDIRA */
/* Dacă tot e invers, înlocuiește MADCTL_MX cu MADCTL_MY */
rotation = 0;

/* Setări standard pentru fereastră */
_width = 128;
_height = 128;
colstart = 0;
rowstart = 0;
break;
}
}

void ST7735_FillRect(uint16_t x, uint16_t y, uint16_t w, uint16_t h, uint16_t
color) {
if((x >= _width) || (y >= _height)) return;
if((x + w - 1) >= _width) w = _width - x;
if((y + h - 1) >= _height) h = _height - y;

ST7735_SetAddressWindow(x, y, x + w - 1, y + h - 1);
uint8_t data[2] = { color >> 8, color & 0xFF };

DC_DATA();
CS_LOW();
for(uint32_t i = 0; i < w * h; i++) {
HAL_SPI_Transmit(&hspi1, data, 2, 100);
}
CS_HIGH();
}

void ST7735_FillScreen(uint16_t color) {
ST7735_FillRect(0, 0, _width, _height, color);
}

void ST7735_DrawPixel(uint16_t x, uint16_t y, uint16_t color) {
ST7735_FillRect(x, y, 1, 1, color);
}

```



```

void ST7735_DrawChar(uint16_t x, uint16_t y, char ch, FontDef font, uint16_t
color, uint16_t bgcolor) {
uint32_t i, b, j;

ST7735_SetAddressWindow(x, y, x + font.width - 1, y + font.height - 1);

DC_DATA();
CS_LOW();

for (i = 0; i < font.height; i++) {
b = font.data[(ch - 32) * font.height + i];

for (j = 0; j < font.width; j++) {

if ((b >> (font.width - 1 - j)) & 0x01) {
uint8_t d[] = { color >> 8, color & 0xFF };
HAL_SPI_Transmit(&hspi1, d, 2, 100);
} else {
uint8_t d[] = { bgcolor >> 8, bgcolor & 0xFF };
HAL_SPI_Transmit(&hspi1, d, 2, 100);
}
}
}
CS_HIGH();
}

void ST7735_WriteString(uint16_t x, uint16_t y, const char *str, FontDef font,
uint16_t color, uint16_t bgcolor) {
while (*str) {
if (x + font.width >= _width) {
x = 0; y += font.height;
if (y + font.height >= _height) break;
if (*str == ' ') { str++; continue; }
}
ST7735_DrawChar(x, y, *str, font, color, bgcolor);
x += font.width;
str++;
}
}

```